CTE và `WITH RECURSIVE`

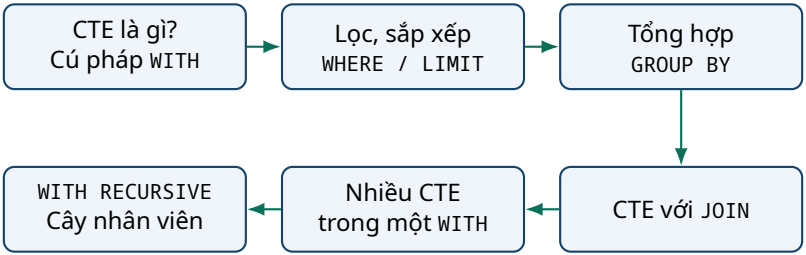
Cú pháp – Phân tích – Ví dụ `classicmodels` – Bài tập

DBMS Course | MySQL 8+ | 2026

Mục tiêu học tập

- Giải thích CTE là gì, phạm vi sử dụng và lợi ích khi chia truy vấn.
- Viết `WITH` kết hợp `WHERE`, `ORDER BY`, `LIMIT`.
- Tạo CTE có `GROUP BY` và nối kết quả với các bảng khác.
- Khai báo nhiều CTE để xây dựng báo cáo theo từng bước.
- Viết `WITH RECURSIVE` duyệt cây quản lý nhân viên.
- Phân tích lỗi và hạn chế của CTE, nhất là CTE đệ quy.

Lộ trình bài học



Các bảng trong classicmodels

Bảng	Vai trò
`customers`	Khách hàng
`orders`	Đơn hàng
`orderdetails`	Chi tiết từng sản phẩm đặt hàng
`payments`	Thanh toán
`products`	Sản phẩm
`productlines`	Dòng sản phẩm
`employees`	Nhân viên và quản lý
`offices`	Văn phòng

Bộ slide sử dụng **tên cột của schema classicmodels**; số liệu trong những bảng ví dụ nhỏ được ghi rõ là minh họa.

Các khóa dùng nhiều trong bài

Bảng nguồn	Cột liên kết	Bảng đích
orders	customerNumber	customers
payments	customerNumber	customers
orderdetails	orderNumber	orders
orderdetails	productCode	products
customers	salesRepEmployeeNumber	employees
employees	reportsTo	employees

Với cây quản lý

`employees.reportsTo` trỏ đến `employees.employeeNumber`: đây là quan hệ **tự tham chiếu**.

Khởi động MySQL: kiểm tra schema

```sql
USE classicmodels;
SHOW TABLES;
DESC customers;
DESC payments;
DESC orders;
DESC orderdetails;
DESC employees;
```

- `USE`: chọn CSDL.
- `DESC`: đọc tên cột, kiểu dữ liệu, khóa và khả năng `NULL`.
- Không cần tự đoán tên cột khi viết `SELECT` hoặc `JOIN`.

CTE là gì?

- CTE = *Common Table Expression*: biểu thức bảng được đặt tên.
- Được khai báo bằng `WITH` ngay trước câu lệnh dùng nó.
- Có thể tham chiếu tên CTE trong truy vấn chính như tham chiếu bảng.
- CTE thông thường chỉ tồn tại trong **phạm vi một câu lệnh**, không tạo bảng vĩnh viễn.

Điểm cần nhớ

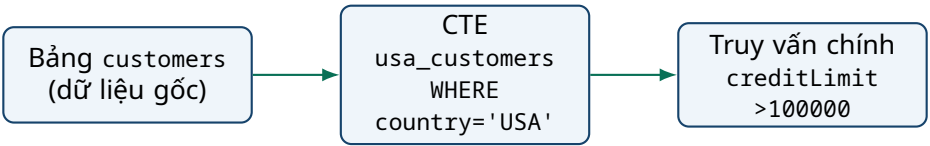
CTE giúp phân tách bài toán thành các bước có tên rõ nghĩa; **không mặc nhiên nhanh hơn** subquery hoặc JOIN.

Cú pháp CTE thường

```sql
WITH cte_name AS (
    SELECT col1, col2, ...
    FROM source_table
    WHERE condition
)
SELECT col1, col2
FROM cte_name;
```

- `WITH`: bắt đầu khai báo CTE.
- `cte_name`: tên kết quả trung gian.
- `AS (SELECT ...)`: định nghĩa kết quả mà CTE biểu diễn.
- `SELECT ... FROM cte_name`: câu truy vấn chính sử dụng CTE.

CTE được sử dụng như thế nào?



Chú ý

Sơ đồ mô tả **logic truy vấn**; optimizer của MySQL có thể gộp hoặc thực thi CTE theo kế hoạch khác.

Ví dụ 1: khách hàng ở USA

```sql
WITH usa_customers AS (
    SELECT customerNumber, customerName,
           city, country, creditLimit
    FROM customers
    WHERE country = 'USA'
)
SELECT customerNumber, customerName,
       creditLimit
FROM usa_customers;
```

- CTE `usa_customers` chứa những dòng của `customers` ở USA.
- Các cột trong SELECT cuối lấy từ `usa_customers`.
- Có thể thay danh sách cột bằng `SELECT *` khi khám phá dữ liệu.

Ví dụ 2: lọc thêm sau CTE

```sql
WITH usa_customers AS (
    SELECT customerNumber, customerName,
           country, creditLimit
    FROM customers
    WHERE country = 'USA'
)
SELECT customerName, creditLimit
FROM usa_customers
WHERE creditLimit > 100000;
```

- Điều kiện trong CTE: `country = 'USA'`.
- Điều kiện ngoài CTE: `creditLimit` > 100000.
- Có thể đọc truy vấn thành hai bước logic: *chọn khách USA, rồi lọc hạn mức*.

Ví dụ 3: sản phẩm Classic Cars

```sql
WITH classic_cars AS (
    SELECT productCode, productName,
           productLine, MSRP
    FROM products
    WHERE productLine = 'Classic Cars'
)
SELECT productCode, productName, MSRP
FROM classic_cars
ORDER BY MSRP DESC;
```

- Bảng nguồn: `products`.
- Cột điều kiện: `products.productLine`.
- Cột kết quả: `productCode`, `productName`, `MSRP`.

Ví dụ 4: CTE đơn hàng đã giao

```sql
WITH shipped_orders AS (
    SELECT orderNumber, orderDate,
           shippedDate, status, customerNumber
    FROM orders
    WHERE status = 'Shipped'
)
SELECT orderNumber, orderDate, customerNumber
FROM shipped_orders
WHERE orderDate >= '2005-01-01';
```

- CTE được tạo từ `orders`.
- Điều kiện ngày dùng so sánh với giá trị kiểu ngày (`DATE`).
- Trong dữ liệu thực tế cần hiểu rõ ý nghĩa của `orderDate` và `shippedDate`.

CTE thường, bảng dẫn xuất và VIEW

CTE

- Có tên ngay trong câu lệnh.
- Dùng lại nhiều lần trong câu lệnh đó.
- Không tồn tại sau khi câu lệnh kết thúc.

Derived table / VIEW

- Derived table: subquery trong `FROM`, cần bí danh.
- VIEW: đối tượng lược đồ được lưu bằng `CREATE VIEW`.
- Không đồng nhất với việc tạo bảng vật lý.

Chọn hình thức nào tùy mục tiêu: dễ đọc, khả năng tái sử dụng, và cách optimizer xử lý.

CTE với ORDER BY và LIMIT

```sql
WITH motorcycles AS (
    SELECT productCode, productName, MSRP
    FROM products
    WHERE productLine = 'Motorcycles'
)
SELECT productCode, productName, MSRP
FROM motorcycles
ORDER BY MSRP DESC
LIMIT 5;
```

- `ORDER BY MSRP DESC`: giá đề xuất giảm dần.
- `LIMIT 5`: chỉ trả về 5 dòng đầu theo thứ tự đó.
- Thêm tiêu chí phụ, chẳng hạn `productCode`, nếu muốn phá hòa một cách nhất quán.

CTE lọc theo năm 2004: payments

```sql
WITH payments_2004 AS (
    SELECT customerNumber, checkNumber,
           paymentDate, amount
    FROM payments
    WHERE paymentDate >= '2004-01-01'
      AND paymentDate <  '2005-01-01'
)
SELECT checkNumber, paymentDate, amount
FROM payments_2004
ORDER BY amount DESC
LIMIT 5;
```

- Cách lọc **nửa khoảng** thường dễ hiểu và áp dụng tốt cho cột ngày/giờ.
- `amount` là cột gốc của `payments`; không phải giá trị tổng hợp.

CTE tồn kho thấp

```sql
WITH low_stock AS (
    SELECT productCode, productName,
           quantityInStock
    FROM products
    WHERE quantityInStock < 1000
)
SELECT productCode, productName,
       quantityInStock
FROM low_stock
ORDER BY quantityInStock ASC;
```

- Tên CTE mô tả ý nghĩa điều kiện trong truy vấn con.
- Kết quả có thể trống nếu CSDL hiện tại không có sản phẩm nào dưới ngưỡng.

Thực hành 1: top 5 sản phẩm

Yêu cầu

Dùng một CTE để lấy **5 sản phẩm Motorcycles có MSRP cao nhất**. Cột cần trả: `productCode`, `productName`, `MSRP`.

Gợi ý làm bài

CTE đọc từ `products`; truy vấn chính dùng `ORDER BY MSRP DESC LIMIT 5`.

Đáp án tham khảo (rút gọn):

```sql
WITH mc AS (
  SELECT productCode, productName, MSRP
  FROM products WHERE productLine='Motorcycles'
)
SELECT * FROM mc ORDER BY MSRP DESC LIMIT 5;
```

Vì sao dùng CTE để tổng hợp?

- Tách phần *tính toán theo nhóm* khỏi phần trình bày hoặc lọc kết quả.
- Giúp đưa `SUM`, `COUNT`, `AVG` vào một kết quả trung gian có tên.
- Dễ nối tổng hợp với `customers`, `orders`, `products`.
- Tên cột suy ra từ hàm tổng hợp cần đặt bằng `AS`, ví dụ `totalPayment`.

Điểm cần nhớ

Một dòng của CTE tổng hợp có thể đại diện cho **một khách hàng / một đơn hàng / một sản phẩm**, không còn là một dòng giao dịch.

Cú pháp: tổng hợp bên trong CTE

```
WITH summary AS (
    SELECT group_key,
           SUM(amount_column) AS total_amount
    FROM source_table
    GROUP BY group_key
)
SELECT group_key, total_amount
FROM summary
WHERE total_amount > threshold;
```

- `group_key`: khóa định danh mỗi nhóm.
- `SUM`: tạo giá trị tổng hợp cho nhóm.
- Bên ngoài CTE có thể lọc trực tiếp cột đã đặt bí danh.

Tổng thanh toán theo khách hàng

```sql
WITH payment_summary AS (
    SELECT customerNumber,
           SUM(amount) AS totalPayment
    FROM payments
    GROUP BY customerNumber
)
SELECT customerNumber, totalPayment
FROM payment_summary
ORDER BY totalPayment DESC;
```

- `payments.amount`: số tiền của từng khoản thanh toán.
- `GROUP BY customerNumber`: một dòng / khách hàng có thanh toán.
- `totalPayment`: cột **tính toán**, không có sẵn trong `payments`.

Từ giao dịch sang kết quả tổng hợp

payments (minh họa)

customerNumber	amount
101	200
101	350
102	180
102	120

payment_summary (minh họa)

customerNumber	totalPayment
101	550
102	300

Chuyển đổi số dòng

`GROUP BY customerNumber` gom nhiều bản ghi thanh toán thành một nhóm; `SUM(amount)` tính tổng của từng nhóm.

Doanh thu từng đơn hàng

```sql
WITH order_summary AS (
    SELECT orderNumber,
           SUM(quantityOrdered * priceEach)
             AS orderTotal
    FROM orderdetails
    GROUP BY orderNumber
)
SELECT orderNumber, orderTotal
FROM order_summary
WHERE orderTotal > 50000
ORDER BY orderTotal DESC;
```

- `quantityOrdered`, `priceEach` thuộc bảng `orderdetails`.
- `orderTotal`: tổng `quantityOrdered * priceEach` của các dòng cùng `orderNumber`.
- Điều kiện ngưỡng áp dụng sau khi đã có CTE tổng hợp.

Số đơn hàng theo khách hàng

```sql
WITH order_count AS (
    SELECT customerNumber,
           COUNT(*) AS numberOfOrders
    FROM orders
    GROUP BY customerNumber
)
SELECT customerNumber, numberOfOrders
FROM order_count
WHERE numberOfOrders > 5;
```

- Bảng `orders`: mỗi hàng tương ứng một đơn hàng.
- `COUNT(*)`: đếm số đơn hàng của mỗi khách hàng.
- Không có hàng tổng hợp cho khách chưa từng đặt hàng (nếu không nối với bảng khách hàng).

Thực hành 2: tổng tiền theo đơn hàng

Yêu cầu

Cho `orderdetails(orderNumber, productCode, quantityOrdered, priceEach)`. Tính `orderTotal` cho từng `orderNumber` bằng CTE.

Gợi ý làm bài

Gom nhóm theo `orderNumber`; tính `SUM(quantityOrdered * priceEach)`.

Đáp án tham khảo (rút gọn):

```sql
WITH order_summary AS (
  SELECT orderNumber,
    SUM(quantityOrdered*priceEach) AS orderTotal
  FROM orderdetails GROUP BY orderNumber
)
SELECT * FROM order_summary;
```

Khi nào JOIN với CTE?

- CTE giữ **chỉ số tổng hợp**; bảng gốc giữ **thông tin mô tả**.
- Khóa của CTE tổng hợp phải đủ để nối, ví dụ `customerNumber`.
- Dùng `INNER JOIN` khi chỉ muốn các thực thể có dữ liệu tổng hợp.
- Dùng `LEFT JOIN` nếu muốn giữ thực thể chưa phát sinh giao dịch.

Điểm cần nhớ

Tránh nối dữ liệu chi tiết với dữ liệu tổng hợp theo khóa không duy nhất, vì dễ nhân số dòng và làm sai tổng.

CTE tổng thanh toán + customers

```sql
WITH payment_summary AS (
    SELECT customerNumber,
           SUM(amount) AS totalPayment
    FROM payments
    GROUP BY customerNumber
)
SELECT c.customerNumber,
       c.customerName, ps.totalPayment
FROM customers AS c
JOIN payment_summary AS ps
  ON c.customerNumber = ps.customerNumber;
```

- `customerName`: cột nguồn `customers`.
- `totalPayment`: cột tổng hợp của CTE `payment_summary`.
- `ON`: hai phía nối cùng khóa `customerNumber`.

CTE doanh thu + đơn hàng + khách hàng

```sql
WITH order_summary AS (
    SELECT orderNumber,
           SUM(quantityOrdered * priceEach) AS orderTotal
    FROM orderdetails
    GROUP BY orderNumber
)
SELECT o.orderNumber, o.orderDate,
       c.customerName, os.orderTotal
FROM orders AS o
JOIN customers AS c
  ON o.customerNumber = c.customerNumber
JOIN order_summary AS os
  ON o.orderNumber = os.orderNumber;
```

- `orders`: mã đơn, ngày đặt, mã khách.
- `customers`: tên khách.
- `order_summary`: tổng tiền theo từng đơn.

INNER JOIN hay LEFT JOIN với CTE?

INNER JOIN

- Chỉ giữ khách hàng có bản ghi tổng hợp phù hợp.
- Khách không có thanh toán không xuất hiện.

LEFT JOIN

- Giữ tất cả khách hàng từ `customers`.
- `totalPayment` là `NULL` nếu không có thanh toán.
- Có thể dùng `COALESCE(...,0)` khi cần hiển thị 0.

LEFT JOIN + COALESCE: cả khách chưa thanh toán

```sql
WITH pay AS (
    SELECT customerNumber,
           SUM(amount) AS totalPayment
    FROM payments GROUP BY customerNumber
)
SELECT c.customerNumber, c.customerName,
       COALESCE(pay.totalPayment, 0)
         AS totalPayment
FROM customers AS c
LEFT JOIN pay
  ON c.customerNumber = pay.customerNumber;
```

- `COALESCE(..., 0)` biến `NULL` (không có giá trị tổng hợp) thành 0 để hiển thị.
- **Phân biệt:** không có thanh toán và thanh toán tổng bằng 0 có thể có ý nghĩa nghiệp vụ khác nhau.

Thực hành 3: JOIN khách hàng

Yêu cầu

Dùng CTE `payment_summary` để trả: `customerNumber`, `customerName`, `totalPayment`.

Gợi ý làm bài

CTE từ `payments`, sau đó `JOIN customers` trên `customerNumber`.

Đáp án tham khảo (rút gọn):

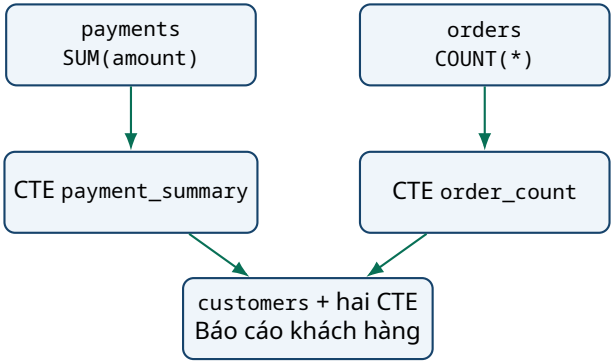
```sql
WITH ps AS (
  SELECT customerNumber, SUM(amount) AS totalPayment
  FROM payments GROUP BY customerNumber
)
SELECT c.customerNumber,c.customerName,ps.totalPayment
FROM customers c JOIN ps
ON c.customerNumber=ps.customerNumber;
```

Cú pháp nhiều CTE

```
WITH
    cte1 AS (
        SELECT ...
    ),
    cte2 AS (
        SELECT ... FROM cte1
    ),
    cte3 AS (
        SELECT ...
    )
SELECT ...
FROM cte2 JOIN cte3 ON ...;
```

- Các định nghĩa CTE được ngăn nhau bằng **dấu phẩy**.
- CTE phía sau có thể tham chiếu CTE định nghĩa trước.
- Truy vấn chính xuất hiện sau định nghĩa CTE cuối cùng.

Hai CTE: báo cáo khách hàng



Hai CTE: khai báo và tính toán

```sql
WITH
payment_summary AS (
    SELECT customerNumber,
           SUM(amount) AS totalPayment
    FROM payments GROUP BY customerNumber
),
order_count AS (
    SELECT customerNumber,
           COUNT(*) AS numberOfOrders
    FROM orders GROUP BY customerNumber
)
-- SELECT chinh nam o slide tiep theo
```

- Hai CTE đều có khóa `customerNumber`.
- Không nối thẳng `payments` với `orders` để tránh nhân số bản ghi.

Hai CTE: truy vấn cuối

```sql
-- Tiep sau phan WITH o slide truoc
SELECT c.customerNumber, c.customerName,
       ps.totalPayment, oc.numberOfOrders
FROM customers AS c
LEFT JOIN payment_summary AS ps
  ON c.customerNumber = ps.customerNumber
LEFT JOIN order_count AS oc
  ON c.customerNumber = oc.customerNumber;
```

- Đầu ra: mã khách, tên khách, tổng thanh toán, số đơn hàng.
- **Đây là phần tiếp nối của cùng một câu lệnh WITH**; không chạy đoạn này riêng lẻ.
- `LEFT JOIN`: giữ cả khách hàng chưa có đơn hoặc thanh toán.

Hai CTE xâu chuỗi: CTE 2 sử dụng CTE 1

```sql
WITH order_totals AS (
  SELECT orderNumber,
         SUM(quantityOrdered*priceEach) AS total
  FROM orderdetails GROUP BY orderNumber
),
high_value AS (
  SELECT orderNumber, total
  FROM order_totals WHERE total > 50000
)
SELECT *
FROM high_value
ORDER BY total DESC;
```

- `order_totals`: tính tổng của từng đơn.
- `high_value`: chọn đơn có tổng vượt ngưỡng.
- Mô hình tách thành các bước có tên dễ đọc.

Thực hành 4: hai CTE tổng hợp

Yêu cầu

Tạo `payment_summary` và `order_count`; nối với `customers` và hiển thị `customerName`, `totalPayment`, `numberOfOrders`.

Gợi ý làm bài

`payment_summary` từ `payments`; `order_count` từ `orders`. Cả hai dùng `customerNumber`.

Khi nào cần WITH RECURSIVE?

- Dữ liệu có cấu trúc nhiều cấp, chẳng hạn tổ chức nhân sự hoặc danh mục cha – con.
- Một hàng có thể tham chiếu hàng khác cùng bảng.
- Không biết trước số cấp phải duyệt; cần tìm mức kế tiếp từ kết quả mức trước.
- `WITH RECURSIVE` có thể mô tả quá trình mở rộng theo từng vòng.

Điểm cần nhớ

Trong `classicmodels`: `employees.reportsTo` liên kết đến `employees.employeeNumber`.

Cấu trúc chuẩn của CTE đệ quy

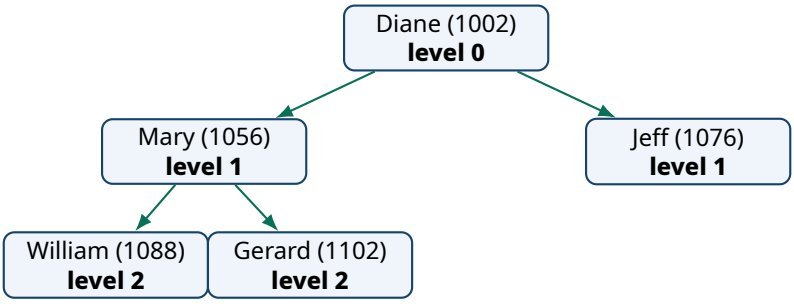
```sql
WITH RECURSIVE hierarchy AS (
    -- Anchor member: diem bat dau
    SELECT ...
    FROM source_table
    WHERE root_condition

    UNION ALL

    -- Recursive member: them cap ke tiep
    SELECT ...
    FROM source_table AS child
    JOIN hierarchy AS parent ON ...
)
SELECT * FROM hierarchy;
```

- **Anchor**: tạo tập kết quả ban đầu.
- **Recursive member**: tham chiếu chính CTE và tạo dòng tiếp theo.
- `UNION ALL`: kết hợp các dòng mới; cần điều kiện kết thúc hợp lý.

Cây quản lý minh họa



Lưu ý: tên và mã nhân viên trong hình là ví dụ minh họa cho cấu trúc `reportsTo`; thứ tự các cấp được tính từ gốc.

employees: dữ liệu minh họa

employeeNumber	firstName	reportsTo
1002	Diane	NULL
1056	Mary	1002
1076	Jeff	1002
1088	William	1056
1102	Gerard	1056

Bản ghi gốc thỏa `reportsTo IS NULL`. Một nhân viên ở cấp $k+1$ có `reportsTo` bằng `employeeNumber` của nhân viên ở cấp k.

CTE đệ quy: phần anchor

```sql
WITH RECURSIVE employee_hierarchy AS (
  SELECT employeeNumber, firstName,
         lastName, reportsTo, jobTitle,
         0 AS level
  FROM employees
  WHERE reportsTo IS NULL

  UNION ALL
  -- Recursive member o slide tiep theo
)
SELECT * FROM employee_hierarchy;
```

- `reportsTo IS NULL`: xác định nhân viên gốc.
- `0 AS level`: cấp gốc.
- Đoạn code này là **minh họa cấu trúc chưa hoàn chỉnh**; xem tiếp phần đệ quy.

CTE đệ quy: phần recursive member

```sql
-- Recursive member ben trong CTE
SELECT e.employeeNumber, e.firstName,
       e.lastName, e.reportsTo, e.jobTitle,
       eh.level + 1 AS level
FROM employees AS e
JOIN employee_hierarchy AS eh
  ON e.reportsTo = eh.employeeNumber
```

- Nhân viên `e` là **con** của nhân viên đã có trong `eh`.
- `eh.level + 1`: cấp mới = cấp của quản lý + 1.
- Mỗi vế của `UNION ALL` phải có cùng số cột và kiểu tương thích.

Truy vấn hoàn chỉnh: cây quản lý

```
WITH RECURSIVE h AS (
  SELECT employeeNumber, firstName, lastName,
         reportsTo, 0 AS level
  FROM employees WHERE reportsTo IS NULL
  UNION ALL
  SELECT e.employeeNumber, e.firstName,
         e.lastName, e.reportsTo, h.level+1
  FROM employees AS e
  JOIN h ON e.reportsTo=h.employeeNumber
)
SELECT employeeNumber, firstName,
       lastName, level FROM h
ORDER BY level, employeeNumber;
```

- Đầu ra có `employeeNumber`, `firstName`, `lastName`, `level`.
- `ORDER BY level, employeeNumber` sắp xếp kết quả sau khi tạo phân cấp.

CTE đệ quy chạy theo các vòng như thế nào?

Vòng	Thành viên vừa tìm được	Giải thích
0	`Diane / 1002 / level 0`	Anchor: `reportsTo IS NULL`.
1	`Mary, Jeff / level 1`	Các nhân viên có `reportsTo=1002`.
2	`William, Gerard / level 2`	Tìm nhân viên báo cáo cho người ở vòng 1.
3	Không thêm dòng mới	Kết thúc, nếu không còn cấp dưới.

Diễn giải logic

Ở mỗi bước, thành viên đệ quy thêm các nhân viên thuộc cấp kế tiếp, đến khi không còn bản ghi mới được sinh ra.

Lọc theo level sau CTE

```sql
WITH RECURSIVE h AS (
  SELECT employeeNumber, reportsTo,
         0 AS level
  FROM employees WHERE reportsTo IS NULL
  UNION ALL
  SELECT e.employeeNumber, e.reportsTo,
         h.level + 1
  FROM employees e
  JOIN h ON e.reportsTo=h.employeeNumber
)
SELECT employeeNumber, reportsTo, level
FROM h WHERE level >= 2;
```

- Điều kiện ở ngoài `WHERE level >= 2` chỉ ẩn các cấp 0–1 khỏi kết quả cuối.
- Nó không thay anchor hay điều kiện duyệt phân cấp.

Giới hạn độ sâu để tăng an toàn

```
WITH RECURSIVE h AS (
  SELECT employeeNumber, reportsTo,
         0 AS level
  FROM employees WHERE reportsTo IS NULL
  UNION ALL
  SELECT e.employeeNumber, e.reportsTo,
         h.level+1
  FROM employees e JOIN h
    ON e.reportsTo=h.employeeNumber
  WHERE h.level < 5
)
SELECT * FROM h;
```

- Đặt điều kiện `h.level < 5` ngay trong **recursive member**.
- Bảo đảm không duyệt sâu hơn `level 5` (nếu gốc là cấp 0).
- Không thay thế nhu cầu kiểm tra chu trình trong dữ liệu.

Điều gì làm CTE đệ quy dừng lại?

- Thành viên đệ quy không tạo thêm dòng nào **hoặc** giới hạn đệ quy được áp dụng.
- Nối sai có thể sinh lặp vô hạn / bùng nổ số dòng.
- Dữ liệu có chu trình cần cơ chế phát hiện các node đã đi qua nếu muốn xử lý tổng quát.
- MySQL có biến `cte_max_recursion_depth` giới hạn số mức lặp mặc định (thường là 1000).

Điểm cần nhớ

Đối với cây nhân sự hợp lệ có một gốc, quan hệ `reportsTo` phải tránh chu trình và thường nên có khóa ngoại tự tham chiếu.

Thực hành 5: truy vấn phân cấp

Yêu cầu

Viết CTE đệ quy trả về `employeeNumber`, `firstName`, `lastName`, `jobTitle`, `level` cho cây `employees`.

Gợi ý làm bài

Anchor: `reportsTo IS NULL`; recursive JOIN: `e.reportsTo = h.employeeNumber`; cấp tăng thêm 1.

So sánh các hình thức CTE

Hình thức	Khai báo	Mục đích
CTE thường	`WITH name AS (...)`	Kết quả trung gian
Nhiều CTE	`WITH a AS (...), b AS (...)`	Chia nhiều bước
CTE đệ quy	`WITH RECURSIVE name AS (...)`	Cấu trúc phân cấp

Có thể kết hợp với `WHERE`, `JOIN`, `GROUP BY`, `ORDER BY` và `LIMIT`.

Lỗi 1: thiếu dấu phẩy giữa hai CTE

```sql
-- SAI: thieu dau phay giua a va b
WITH a AS (SELECT * FROM orders)
b AS (SELECT * FROM customers)
SELECT * FROM a;

-- DUNG: co dau phay
WITH a AS (SELECT * FROM orders),
     b AS (SELECT * FROM customers)
SELECT * FROM a;
```

- Trong một `WITH` có nhiều CTE, ngăn cách bằng **dấu phẩy**.
- Mỗi CTE có một tên và một định nghĩa trong dấu ngoặc tròn.

Lỗi 2: quên câu truy vấn chính

```sql
-- SAI: WITH khong the ket thuc mot minh
WITH usa_customers AS (
  SELECT * FROM customers
  WHERE country='USA'
);

-- DUNG: them truy van su dung CTE
WITH usa_customers AS (
  SELECT * FROM customers
  WHERE country='USA'
)
SELECT * FROM usa_customers;
```

- `WITH` không phải lệnh tạo bảng riêng lẻ.
- CTE phải được theo sau bởi một câu lệnh hợp lệ sử dụng nó.

Lỗi 3: tên CTE trùng tên bảng

```sql
-- Kho doc va co the gay nham lan
WITH customers AS (
  SELECT * FROM customers
  WHERE country='USA'
)
SELECT * FROM customers;

-- Nen dat ten mo ta nghiep vu
WITH usa_customers AS (
  SELECT * FROM customers
  WHERE country='USA'
)
SELECT * FROM usa_customers;
```

- Dùng tên riêng mô tả chức năng CTE.
- Tránh đụng tên bảng gốc hoặc tên CTE đã tồn tại trong cùng phạm vi.

Lỗi 4: điều kiện JOIN sai trong CTE đệ quy

```
-- DUNG: child.reportsTo = parent.employeeNumber
JOIN employee_hierarchy AS parent
  ON e.reportsTo = parent.employeeNumber

-- SAI trong bai toan nay:
JOIN employee_hierarchy AS parent
  ON e.employeeNumber = parent.employeeNumber
```

- Nếu tự nối theo `employeeNumber` ở cả hai phía, truy vấn không đi sang con.
- Điều kiện đúng phải liên kết `reportsTo` của nhân viên với mã người quản lý.

Lỗi 5: UNION ALL vs UNION

```
WITH RECURSIVE h AS (
  SELECT employeeNumber, reportsTo,
         0 AS level
  FROM employees WHERE reportsTo IS NULL
  UNION ALL
  SELECT e.employeeNumber, e.reportsTo,
         h.level+1
  FROM employees e
  JOIN h ON e.reportsTo=h.employeeNumber
)
SELECT * FROM h;
```

- `UNION ALL` giữ các dòng sinh ra; thường phù hợp với cây đơn giản.
- `UNION` loại dòng trùng, nhưng không bảo đảm loại được chu trình nếu `level` liên tục tăng.
- Không tự động coi `UNION` là cơ chế chống vòng lặp an toàn.

Các lưu ý về hiệu năng

- CTE ưu tiên khả năng diễn đạt; không mặc định giúp tăng hiệu năng.
- Lọc các cột/dòng cần thiết và đặt chỉ mục trên khóa JOIN được sử dụng nhiều.
- `EXPLAIN` để quan sát kế hoạch thực thi thay vì suy đoán.
- Với truy vấn phân cấp lớn: giới hạn độ sâu, kiểm tra chu trình, và tránh mở rộng không cần thiết.

Điểm cần nhớ

Ở MySQL, việc optimizer gộp hay materialize CTE tùy trường hợp và phiên bản; không nên khẳng định CTE luôn là bảng tạm vật lý.

Tự luyện A: tổng thanh toán

Yêu cầu

Dùng CTE đếm số khoản thanh toán và tính tổng tiền theo `customerNumber`; chỉ trả các khách có tổng tiền trên 50000.

Gợi ý làm bài

Bảng nguồn `payments`; cột sử dụng `customerNumber`, `amount`; đặt bí danh cho `COUNT(*)` và `SUM(amount)`.

Tự luyện B: top doanh thu sản phẩm

Yêu cầu

Từ `orderdetails`, tạo CTE tính doanh thu theo `productCode`; nối với `products` để lấy `productName`.

Gợi ý làm bài

Doanh thu từng dòng = `quantityOrdered * priceEach`. Gom theo `productCode`, sau đó JOIN theo `productCode`.

Tự luyện C: nhiều CTE

Yêu cầu

Hiển thị `customerName`, `numberOfOrders`, `totalPayment`; giữ cả khách hàng chưa từng thanh toán.

Gợi ý làm bài

Tạo 2 CTE riêng từ `orders` và `payments`; dùng `customers LEFT JOIN ...`.

Tự luyện D: quản lý nhân viên

Yêu cầu

Viết CTE đệ quy cho cấp quản lý và chỉ hiển thị nhân viên ở cấp ít nhất 2.

Gợi ý làm bài

Giữ phần anchor và recursive member đầy đủ; đặt `WHERE level >= 2` ở truy vấn chính.

Tổng kết: 6 ý chính

1. CTE là **kết quả trung gian có tên** trong phạm vi câu lệnh.
2. Cú pháp cơ bản: `WITH name AS (SELECT ...)`.
3. CTE kết hợp tự nhiên với `WHERE`, `ORDER BY`, `LIMIT`.
4. Tổng hợp `GROUP BY` trong CTE rồi `JOIN` với bảng mô tả.
5. Nhiều CTE giúp chia báo cáo thành các bước rõ ràng.
6. `WITH RECURSIVE` gồm **anchor + recursive member**; dùng duyệt phân cấp.

Tự học

Lab tương tác SELECT 6: làm đủ 45 bài, xem schema và thử `Run / Check` cho các truy vấn.

Tài liệu và bài tập

Nguồn bài giảng:

- `https://vudmvn.github.io/dbms-course/lectures/MySQL/querying-data/select/select-statement-6/`

Các kỹ năng nên đạt:

1. Xác định cột nguồn trước khi viết `SELECT`.
2. Xác định khóa nhóm hoặc khóa nối trước khi viết CTE.
3. Phân biệt câu lệnh cuối với các định nghĩa CTE.
4. Duyệt đúng cây quản lý, nhận diện anchor/recursive member.